Author: Andrey Pautov. Published on Medium

CTI Research: Kubernetes & Cloud-Native Threat Landscape

Technical Kill Chain Analysis, Detection Engineering, and Defensive Architecture (2023 – Q2 2026)

Classification: TLP:WHITE — Unrestricted Distribution
Author: Andrey Pautov
Date: April 23, 2026
Assessment window: 2023 – Q2 2026
Evidence cutoff: April 23, 2026 (UTC)
Audience: CISO, Detection Engineers, SOC Analysts (Tier 2–3), Cloud Security Architects
Primary Sources: Sysdig TRT, Aqua Nautilus, Palo Alto Unit 42, Microsoft Threat Intelligence, CrowdStrike, NVD, AWS/Azure/GCP Documentation, SigmaHQ, Falco, Tetragon, CNCF

Table of Contents

- Introduction
- Purpose
- Scope
- Structure
- How to Use This Document
- Methodology
- Evidence Labels and Confidence
- Executive Summary
- Pillar 1: Threat Actor Landscape & Kill Chains
- 1.1 Threat Actor Taxonomy
- 1.2 SCARLETEEL — Cloud Intrusion via Containerized Workload
- 1.3 TeamTNT RBAC Buster — Kubernetes Backdoor Campaign
- 1.4 Kinsing — Persistent Opportunistic Cryptominer
- 1.5 Slow Pisces / TraderTraitor — DPRK Crypto-Theft via Cloud Identity
- 1.6 FAMOUS CHOLLIMA — DPRK Fake-Worker / Insider-Access Cluster
- 1.7 LLMjacking — Cloud Credential Abuse Against AI Services
- 1.8 CVE-2025-55182 React2Shell — Web-App RCE as a Kubernetes Foothold
- 1.9 Defense Evasion: Memory-Only Execution and Log Tampering
- Pillar 2: Telemetry Collection & Visibility Architecture
- 2.1 Kubernetes Audit Log Architecture
- 2.2 eBPF Runtime Telemetry
- 2.3 Telemetry Architecture by Platform
- Pillar 3: Detection Engineering & "Parsers as Code"
- 3.1 Log Normalization and UDM Mapping
- 3.2 Detection Rule 1 — Privileged Pod Creation, Non-System Namespaces (Backend-Dependent Sigma-Style Example)
- 3.2b Detection Rule 1b — Suspicious Privileged DaemonSet in kube-system (Backend-Dependent Sigma-Style Example)
- 3.3 Detection Rule 2 — RBAC ClusterRoleBinding Backdoor (YARA-L)

- 3.4 Detection Rule 3 — exec-then-Outbound / IMDS Access (Elastic EQL)
- 3.5 Detection Rule 4 — Privileged Pod then Host File Access (Sigma Correlation)
- Pillar 4: Defensive Architecture & Hardening
- 4.1 Admission Controllers: Enforce Before Scheduling
- 4.2 Network Policy: Block IMDS and Constrain Egress
- 4.3 Seccomp and AppArmor: Syscall-Level Restriction
- 4.4 RBAC Hardening: Minimize Service Account Token Exposure
- Takeaways for the SOC
- MITRE ATT&CK Mapping
- References

Introduction

Purpose

This report is a practitioner-grade Cyber Threat Intelligence document covering the Kubernetes and cloud-native threat landscape from 2023 through Q2 2026. It is designed for direct operational use: detection engineers building or tuning analytic rules, SOC analysts (Tier 2–3) triaging Kubernetes-related alerts, cloud security architects evaluating control posture, and CISOs reviewing coverage against documented adversary patterns.

The document does not survey Kubernetes security in the abstract. It is grounded in named, sourced threat actor campaigns and real vulnerabilities with confirmed exploitation paths. Every claim carries an explicit evidence label. Every detection rule includes explicit backend dependencies, normalization requirements, and portability caveats. Every defensive control section states what it blocks, what it does not block, and which platform constraints apply. Content presented without a primary source is labeled **Assessed** or **Inferred** and is distinguished from directly observed technical evidence.

Scope

Assessment window: 2023 – Q2 2026
Evidence cutoff: April 23, 2026 (UTC)

The report covers:

- **Threat actor kill chains**: six named clusters (SCARLETEEL, TeamTNT RBAC Buster, Kinsing, Slow Pisces / TraderTraitor, FAMOUS CHOLLIMA, LLMjacking operators) plus CVE-2025-55182 (React2Shell) as a concrete foothold vector, and a cross-actor analysis of defense evasion techniques
- **Telemetry architecture**: Kubernetes API audit logs, eBPF-based runtime telemetry (Falco, Tetragon, Tracee), and platform-specific collection constraints across EKS, AKS, GKE, EKS Auto Mode, and AWS Fargate
- **Detection engineering**: four production-oriented detection rules (Sigma, YARA-L / Chronicle, Elastic EQL) with explicit normalization requirements, backend dependencies, and a Sigma correlation rule; all rules are explicitly labeled by portability and environment-sensitivity
- **Defensive controls**: admission control policy (Kyverno), network policy (Cilium, Kubernetes NetworkPolicy), syscall restriction (Seccomp, AppArmor), RBAC hardening, and OPA Gatekeeper constraint policy

Out of scope: General Kubernetes operational hardening not tied to documented adversary techniques; managed Kubernetes control-plane internals beyond what is observable from audit logs; offensive tooling development.

Structure

The report is organized around four interdependent pillars:

Pillar	Content
Pillar 1 — Threat Actor Landscape & Kill Chains	Named actor profiles with sourced kill chains, ATT&CK mappings, and evidence labels. Establishes what adversaries actually do and what artifacts they leave.
Pillar 2 — Telemetry Collection & Visibility Architecture	What logs and signals exist, how to enable them, where platform constraints create blind spots. Detection rules in Pillar 3 are only meaningful against the collection model described here.
Pillar 3 — Detection Engineering	Four detection rules derived directly from Pillar 1 kill chains. Each rule is paired with its normalization dependencies, backend constraints, and a statement of what it cannot detect.
Pillar 4 — Defensive Architecture & Hardening	Controls that prevent or constrain the techniques in Pillar 1. Each control section is explicit about coverage boundaries and does not overstate prevention where the sample policy provides detection only.

The Takeaways section at the end synthesizes the ten highest-leverage actions derived from the full analysis, ordered by expected impact on detection coverage and adversary friction.

How to Use This Document

Detection engineers should read Pillar 2 before Pillar 3. The detection rules assume a specific telemetry model; deploying rules without the corresponding collection pipeline produces silent failures, not false positives. Each rule's *Assumptions / normalization dependencies* block is the prerequisite checklist.

SOC analysts should use the kill chain sections (1.2–1.9) as behavioral context for alert triage, and the ATT&CK mapping table as a lookup when a technique ID appears in an alert. The Takeaways section provides the prioritized SOC action list.

Security architects should use Pillar 4 in conjunction with the control coverage gaps noted in Pillar 1. The Kyverno and OPA Gatekeeper policies are starting points, not drop-in production controls; each includes an explicit statement of what the sample policy does not enforce.

CISOs can begin with the Executive Summary and Takeaways, then use the ATT&CK mapping table to assess which documented adversary techniques are covered, partially covered, or not addressed by current controls.

Methodology

Source material is drawn exclusively from named primary sources: vendor threat research reports, NVD advisories, and platform provider documentation. No claims are derived from secondary or anonymous reporting without an independent primary source being available. The evidence label model (Observed / Reported / Assessed / Inferred / Claimed) is defined in the next section and applied consistently throughout the document.

Detection rules and defensive configurations were reviewed across multiple revision cycles for technical accuracy, including syntax validation, portability scope, and control-boundary honesty. Rules known to require environment-specific tuning are marked as such and are not presented as generic baselines.

Evidence Labels and Confidence

All substantive claims in this document carry one of the following labels. Inline citations follow every labeled claim.

Label	Meaning
Observed	Technical artifact available in a named primary source: sample, pcap, config, telemetry
Reported	Documented by a named authoritative vendor; full victim-side telemetry not fully public
Assessed	Analytical conclusion synthesized from multiple Observed/Reported items; not standalone proof
Inferred	Logical deduction from established facts; no direct primary source; may be wrong

Label	Meaning
Claimed	Vendor or actor assertion without independent technical validation

Every code example is either syntactically valid for its named target platform or explicitly labeled **[PSEUDOCODE]** or **[PARSER-SPECIFIC EXAMPLE]**. Detection rules include an *Assumptions / normalization dependencies* note. Mitigation sections include *What this blocks*, *What this does not block*, and *Platform caveats*.

Executive Summary

Kubernetes has become the dominant container orchestration platform, and adversarial targeting has grown proportionally. Microsoft Threat Intelligence noted that **51% of workload identities were found completely inactive**, creating a persistent attack surface that threat actors exploit by operating under dormant, unmonitored credentials [R1].

The threat landscape has matured from opportunistic cryptomining toward multi-stage cloud intrusions. SCARLETEEL, documented by Sysdig TRT, pivoted from a Kubernetes pod into AWS IAM and across multiple accounts via a Terraform state file, exfiltrating over 1 TB of proprietary data [R2]. Palo Alto Unit 42 attributed a 2025 cryptocurrency exchange intrusion — in which Kubernetes post-exploitation was a pivot layer — to North Korea's Lazarus Group (tracked as Slow Pisces / TraderTraitor) [R3]. CrowdStrike separately documented FAMOUS CHOLLIMA, a distinct DPRK-linked cluster focused on fraudulent employment rather than technical exploitation [R4]. Sysdig documented LLMjacking: stolen cloud credentials abused against hosted AI inference services, with victim costs reported up to $100,000 per day — a pattern that can intersect Kubernetes environments but is not inherently Kubernetes-native [R5].

Three runc vulnerabilities — CVE-2025-31133, CVE-2025-52565, and CVE-2025-52881 — disclosed in late 2025 demonstrate that the container isolation boundary remains an active research and exploitation target, each under specific local/container-context prerequisites [R6]. The earlier CVE-2024-21626 ("Leaky Vessels") exposed a file-descriptor-leak escape path in runc through v1.1.11 [R7].

Effective detection requires layering Kubernetes API audit logs — which must be explicitly configured, with enablement procedures varying by managed service — with eBPF-based or equivalent kernel-level runtime telemetry (Falco, Tetragon), then normalizing both streams to a common schema before correlation. Neither source alone produces high-confidence detections for the behavioral patterns documented in this report.

Pillar 1: Threat Actor Landscape & Kill Chains

1.1 Threat Actor Taxonomy

Actor	Motivation	Signature Technique	Attribution
SCARLETEEL	Data theft, cryptomining	K8s → AWS IAM pivot via IMDS and Terraform state	Financially motivated intrusion set (Assessed; Sysdig TRT [R2])
TeamTNT / RBAC Buster	Cryptomining, persistence	Exposed API server, ClusterRoleBinding backdoor	Cybercrime cluster (Reported; Aqua Nautilus [R8])
Kinsing	Cryptomining	Exposed Docker API, PostgreSQL RCE, userland rootkit	Cybercrime (Observed/Reported; Microsoft, Aqua [R9])
Slow Pisces / TraderTraitor	Cryptocurrency theft, espionage	K8s identity pivot, cloud credential abuse	DPRK state-sponsored (Attributed; Unit 42 [R3])
FAMOUS CHOLLIMA	Fraudulent employment, insider access	Fake-worker personas, hands-on-keyboard with legitimate tools	DPRK-nexus (Attributed; CrowdStrike [R4])

Actor	Motivation	Signature Technique	Attribution
LLMjacking operators	Cloud resource hijacking	Stolen API keys, cloud credential abuse against AI services	Financially motivated (Assessed; Sysdig [R5])

> ***Note:*** *Slow Pisces/TraderTraitor and FAMOUS CHOLLIMA are distinct DPRK-linked clusters with different tradecraft profiles and should not be merged into a single narrative. They are addressed in separate subsections below.*

1.2 SCARLETEEL — Cloud Intrusion via Containerized Workload

Evidence label: Reported (primary source: Sysdig TRT [R2])

SCARLETEEL is documented by Sysdig TRT as a financially motivated intrusion set — not labeled an APT in the primary reporting. Sysdig reported exfiltration of over **1 TB of proprietary customer data** [R2].

Kill Chain (Reported [R2])

```
Phase 1 — Initial Access: Vulnerable Public-Facing Application (T1190)
  └─ Target: JupyterLab notebook container, public-facing, no authentication
  └─ Data science workloads frequently carry excessive IAM permissions
     and weak or absent authentication

Phase 2 — Execution and Enumeration (T1610, T1083)
  └─ Deploy cryptominer as a cover action
  └─ Enumerate environment variables for cloud credentials
  └─ Deploy Peirates (open-source Kubernetes pentest tool)
  └─ Query Kubernetes API server using the auto-mounted SA token

Phase 3 — Cloud Credential Theft via IMDS (T1552.001, T1552.005)
  └─ HTTP GET to AWS IMDS:
      http://169.254.169.254/latest/meta-data/iam/security-credentials/<role>
  └─ Validate credential validity: aws sts get-caller-identity

Phase 4 — Cloud Privilege Escalation (T1078.004)
  └─ Enumerate EC2, Lambda, IAM using the stolen IAM role
  └─ Discover Terraform state file in an S3 bucket
      → State file contains plaintext credentials and cross-account trust data

Phase 5 — Lateral Movement Across AWS Accounts (T1550.001)
  └─ AssumeRole into connected AWS accounts using Terraform-exposed trusts

Phase 6 — Data Exfiltration and Persistence (T1537, T1098.001)
  └─ Exfiltrate S3 objects, source code, .git-credentials
  └─ Exfiltration via Bash /dev/tcp built-in — does not spawn curl/wget
  └─ Data encoded in memory variable (SEND_B64_DATA), sent to external endpoint
  └─ Persist via Lambda backdoors and secondary IAM user creation
```

Defense evasion TTPs (Observed [R2]):
- Executed `rm /var/log/syslog` repeatedly to destroy agent-based log forwarder data before it was shipped
- Used `history -cw` and `clear` to erase shell forensic trail
- Used Bash `/dev/tcp` built-in rather than `curl` / `wget` to avoid spawning processes that trigger tool-execution behavioral rules

ATT&CK Mappings: `T1190`, `T1610`, `T1083`, `T1552.001`, `T1552.005`, `T1078.004`, `T1537`, `T1098.001`, `T1027`, `T1070.003`

1.3 TeamTNT RBAC Buster — Kubernetes Backdoor Campaign

Evidence label: Reported (primary source: Aqua Nautilus [R8])

Aqua Nautilus documented this campaign using Kubernetes honeypots over a three-month period. 350+ production clusters were found openly accessible; at least 60% were already compromised and running active malware at the time of discovery [R8]. The RBAC Buster technique creates a persistent backdoor that survives remediation of the original misconfiguration.

Kill Chain (Reported [R8])

```
Phase 1 — Initial Access: Exposed Kubernetes API Server (T1190)
  └─ Target: clusters with --anonymous-auth=true or absent RBAC policy
  └─ Discovery: Shodan/Censys queries for ports 6443 (HTTPS), 8080 (legacy)
  └─ Verify unauthenticated access:
      curl -k https://<target>:6443/api/v1/namespaces

Phase 2 — RBAC Backdoor Creation (T1098)
  └─ Create ClusterRoleBinding granting cluster-admin to attacker SA:
      kubectl create clusterrolebinding backdoor \
        --clusterrole=cluster-admin \
        --serviceaccount=kube-system:attacker-sa
  └─ This binding persists even if the original misconfiguration is closed

Phase 3 — Persistence via DaemonSet (T1053.007)
  └─ Deploy DaemonSet with name mirroring kube-proxy
  └─ Container: XMRig Monero miner in kube-system namespace
  └─ hostPID: true, hostNetwork: true, privileged: true

Phase 4 — Defense Evasion (T1036)
  └─ Namespace: kube-system (hides among platform components)
  └─ Image name mimics a legitimate kube-proxy component
  └─ Removes competing malware to maximise resource availability

Phase 5 — Impact (T1496 — Resource Hijacking)
  └─ XMRig configured to pool.hashvault.pro
  └─ No resource limits — maximizes mining throughput
```

Reconstructed DaemonSet manifest (threat intelligence artifact — do not deploy):

```
# Reconstructed from Aqua Nautilus honeypot analysis [R8]
# FOR REFERENCE ONLY
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: kube-proxy-manager
  namespace: kube-system
  labels:
    k8s-app: kube-proxy
spec:
  selector:
    matchLabels:
      k8s-app: kube-proxy
  template:
    spec:
      hostPID: true
      hostNetwork: true
      tolerations:
      - key: node-role.kubernetes.io/master
        effect: NoSchedule
      containers:
      - name: kube-proxy
        image: xmrig/xmrig:latest
```

```
        args: ["--pool=pool.hashvault.pro:443", "--user=<wallet>", "--tls"]
        securityContext:
          privileged: true
        resources: {}
```

ATT&CK Mappings: `T1190`, `T1098`, `T1053.007`, `T1036`, `T1496`

1.4 Kinsing — Persistent Opportunistic Cryptominer

Evidence label: Observed/Reported (primary sources: Microsoft [R9], Aqua Nautilus [R8], Wiz [R7])

Kinsing has been tracked since 2019 and continues to evolve. Its userland rootkit, which hooks `ps`, `ls`, and `netstat` output, is the primary reason kernel-level telemetry is operationally necessary in environments where Kinsing activity is a concern.

Initial access vectors (Observed):
1. Docker/containerd API exposed on TCP 2375 without TLS or authentication
2. PostgreSQL with `trust` authentication in `pg_hba.conf`, leading to `COPY TO/FROM PROGRAM` RCE

CVE-2024-21626 — container escape via runc file descriptor leak:

CVE-2024-21626 is a file-descriptor leak in runc through v1.1.11, disclosed January 2024 [R7]. The vulnerability requires that an attacker control the container image (specifically the `WORKDIR` directive) or the `kubectl exec` arguments, and that the host is running a vulnerable runc version.

> ***Attribution caveat (Assessed):*** *The CVE is presented here because it is relevant to the Kinsing toolset profile and to container runtime hardening generally. Direct evidence that Kinsing specifically weaponized CVE-2024-21626 in confirmed production campaigns is not established in the primary sources reviewed for this paper. The actor-to-CVE linkage should be treated as Assessed, not Observed.*

CVE-2024-21626 escape path (Reported [R7]):

```
Prerequisites:
  - Vulnerable runc version (≤ v1.1.11) on the container host
  - Attacker controls container image or kubectl exec session

Attack path:
  1. Container WORKDIR is set to a leaked runc file descriptor path:
       WORKDIR /proc/self/fd/<leaked_fd>
     The leaked fd references a host directory at container start time.

  2. Container process begins with its CWD resolved to a host path.

  3. Path traversal breaks out of the container root:
       cd ../../../../..

  4. Attacker reads or writes host-sensitive files:
       cat /var/lib/kubelet/kubeconfig
       echo <key> >> /root/.ssh/authorized_keys
```

Mitigation for CVE-2024-21626: Upgrade runc to v1.1.12 or later. Applying a Seccomp profile that restricts relevant mount and procfs syscalls reduces exploitability. Denying `hostPath` volumes does **not** mitigate this vulnerability; the escape uses runc's internal file descriptor, not a mounted host path.

Post-compromise behavior (Observed [R8], [R9]):
- Userland rootkit hooks `ps`, `ls`, `netstat` output to hide miner processes from userspace enumeration tools
- SSH key injection into `~/.ssh/authorized_keys` for persistent host access

- Internal network scanning targeting additional Docker APIs and Kubernetes nodes
- Active removal of competing malware (Kinsing and TeamTNT contest the same hosts)

ATT&CK Mappings: `T1190`, `T1611`, `T1014`, `T1098.004`, `T1046`, `T1496`

1.5 Slow Pisces / TraderTraitor — DPRK Crypto-Theft via Cloud Identity

Evidence label: Reported (primary source: Palo Alto Unit 42 [R3])
Attribution: North Korean state-sponsored; tracked by Unit 42 as Slow Pisces and by FBI/CISA as TraderTraitor [R3].

Unit 42 documented a 2025 intrusion at a cryptocurrency exchange in which a Kubernetes environment served as a post-exploitation pivot layer. Losses were assessed at approximately $1.5 billion in Ethereum [R3]. This campaign illustrates that Kubernetes is targeted as an identity transit layer in DPRK financial operations.

How Kubernetes was used as a pivot layer (Reported [R3]):
- Initial access via spearphishing targeting employees at the cryptocurrency exchange [R3]
- Post-exploitation: adversary deployed a malicious pod into the Kubernetes cluster
- The malicious pod accessed the auto-mounted service account token belonging to a CI/CD automation workload that held elevated cluster privileges
- That SA token was used to authenticate to the kube-apiserver and perform further cluster enumeration
- Enumeration of cloud provider IMDS from inside pods to obtain IAM credentials for the host node role
- Persistence via dormant, legitimately named workloads — no anomalous process behavior to trigger behavioral alerts

Key detection implication (Inferred): Malicious pod deployments by a compromised insider or phished identity, SA token usage from unexpected pod names, and dormant workloads with no baseline API activity all require explicit alert logic against audit log data. These patterns do not generate signals in environments with only runtime telemetry.

ATT&CK Mappings: `T1566` (spearphishing initial access), `T1610` (deploy container — malicious pod), `T1528`, `T1552.005`, `T1078.004`

1.6 FAMOUS CHOLLIMA — DPRK Fake-Worker / Insider-Access Cluster

Evidence label: Reported (primary source: CrowdStrike 2025 Threat Hunting Report [R4])
Attribution: DPRK-nexus, attributed by CrowdStrike [R4].

> *This cluster is operationally distinct from Slow Pisces/TraderTraitor (Section 1.5) and should not be presented as the same campaign or blended kill chain.*

Operational profile (Reported [R4]):
- Operatives create fraudulent professional identities and apply for remote IT contractor and developer roles, gaining legitimate credentials without any technical exploitation
- CrowdStrike's 2025 Threat Hunting Report documented a 220% year-over-year increase in organizations where this insider-persona approach was confirmed [R4]
- The same report documented that 81% of all DPRK-nexus hands-on-keyboard intrusions in the tracked dataset did not deploy custom malware, relying instead on native OS and cloud tooling. This statistic reflects a broad category from the CrowdStrike dataset; it is not presented here as a FAMOUS CHOLLIMA-specific precision metric [R4]

Kubernetes relevance (Inferred): An operative holding legitimate developer credentials can issue `kubectl` commands, read Secrets, list pods, and enumerate RBAC assignments through normal authenticated channels. Audit logs capture these actions, but distinguishing legitimate developer activity from low-and-slow reconnaissance requires behavioral baselining over time.

ATT&CK Mappings: `T1078.001`, `T1078.003`, `T1087.004`, `T1528`

1.7 LLMjacking — Cloud Credential Abuse Against AI Services

Evidence label: Reported (primary source: Sysdig TRT [R5])

> ***Scope:*** *LLMjacking is a cloud credential and API-key abuse pattern. It targets hosted AI inference services (AWS Bedrock, Anthropic Claude API, OpenAI API). It is* ***not inherently a Kubernetes-native attack category****. Its intersection with Kubernetes occurs when compromised credentials originate from Kubernetes-hosted workloads.*

Documented pattern (Reported [R5]):
- Attackers steal cloud credentials or AI provider API keys — from exposed `.env` files, misconfigured object storage, or IMDS credential theft
- Credentials invoke computationally expensive AI models against the victim's billing account
- Attackers monetize by reselling access at below-market rates
- Sysdig reported victim compute costs reaching up to **$100,000 per day** before detection [R5]

Kubernetes-specific entry points (Assessed):
- Kubernetes Secrets containing `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, or cloud IAM credentials scoped to Bedrock/Vertex AI
- IMDS credential theft from nodes running ML inference pods — the same path as SCARLETEEL Phase 3
- Unauthenticated model-serving endpoints exposed as Kubernetes Services

Detection indicator: An unexplained spike in AI API token consumption or cloud compute billing with no corresponding application traffic increase.

1.8 CVE-2025-55182 React2Shell — Web-App RCE as a Kubernetes Foothold

Evidence label: Reported (primary source: Microsoft Security Blog [R10])

CVE-2025-55182 is a pre-authentication remote code execution vulnerability in React Server Components / Next.js, disclosed December 2025. It is a **web application vulnerability**. Its Kubernetes relevance is conditional: when the vulnerable application runs inside a Kubernetes pod, the resulting code execution occurs within that pod's security context.

Vulnerability summary (Reported [R10]):
- Affected component: React Server Components ("Flight" protocol) in Next.js
- Attack type: Crafted HTTP request triggers server-side code execution without authentication
- Exploitability: Remotely exploitable; no credentials required [R10]

The specific exploit mechanism (serialization details, JS engine behavior) is outside the scope of this paper. Defenders should consult the NVD advisory and vendor patch documentation directly.

Kubernetes-specific post-exploitation path (Inferred):

```
Vulnerable React/Next.js app runs as a Kubernetes pod
  → RCE executes within the pod's process context
  → Attacker enumerates /var/run/secrets/ for auto-mounted SA token
  → Attacker queries kube-apiserver with the SA token
  → Further steps follow the cloud-identity pivot path documented in SCARLETEEL
```

Mitigation: Patch Next.js to the version resolving CVE-2025-55182 [R10]. Disable SA token auto-mounting on pods running web frontends that do not require API server access (see Section 4.4).

1.9 Defense Evasion: Memory-Only Execution and Log Tampering

The following techniques appear in primary source post-mortem reporting:

Technique	Actor	Mechanism	Evidence Label
Shell built-in TCP (`/dev/tcp`)	SCARLETEEL	Exfiltrates data via Bash built-in; no `curl` / `wget` process spawned	Observed [R2]
Syslog deletion (`rm /var/log/syslog`)	SCARLETEEL	Destroys agent-based forwarder data before it ships	Observed [R2]
In-memory variable exfiltration	SCARLETEEL	Data encoded in `SEND_B64_DATA` ; no disk artifact written	Observed [R2]
Userland rootkit	Kinsing	Hooks `ps` , `ls` , `netstat` to hide miner processes	Observed [R8]
History clearing (`history -cw`)	SCARLETEEL	Destroys shell forensic trail	Observed [R2]
Native tooling only	DPRK-nexus (FAMOUS CHOLLIMA)	Post-access activity via `kubectl` , `aws` , `gcloud`	Reported [R4]

Architectural implication (Assessed): Detection relying solely on userspace process enumeration or file scanning is structurally blind to Kinsing's rootkit and SCARLETEEL's in-memory exfiltration. Kernel-level telemetry — particularly eBPF-based tools that intercept syscalls before any userspace filtering occurs — is a high-value complement. This does not mean eBPF is the only viable approach; `auditd` with syscall rules, kernel module-based agents, and commercial EDR products with privileged kernel drivers provide comparable visibility with different deployment and performance tradeoffs.

Pillar 2: Telemetry Collection & Visibility Architecture

2.1 Kubernetes Audit Log Architecture

Kubernetes audit logs are the authoritative record of interactions with the kube-apiserver. Without an explicit audit policy, RBAC manipulation, pod exec sessions, and service account token requests are invisible to any downstream detection system.

Audit event lifecycle:

Stage	When Generated	Security Value
`RequestReceived`	On API handler receipt, before authorization	Low — produces duplicates; includes requests that will be rejected
`ResponseStarted`	After response headers sent; long-running requests only	Low for most detection use cases
`ResponseComplete`	After full response body transmitted	Highest — includes authorization outcome and full request/response objects

Omitting `RequestReceived` reduces event volume without losing security-relevant data, because every request that completes also generates a `ResponseComplete` record.

> ***Secrets logging warning:*** *Logging Secrets at `RequestResponse` level writes plaintext credential values into the SIEM ingestion pipeline. The CIS GKE Benchmark v1.5.0 explicitly recommends against this [R11]. Set Secrets to `Metadata` level only.*

Audit Policy Configuration

```
# /etc/kubernetes/audit-policy.yaml
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
  - "RequestReceived"
rules:
  # Service account token requests
  - level: RequestResponse
    verbs: ["create"]
    resources:
      - group: ""
        resources: ["serviceaccounts/token"]
      - group: "rbac.authorization.k8s.io"
        resources:
          - "clusterroles"
          - "clusterrolebindings"
          - "roles"
          - "rolebindings"

  # Interactive access
  - level: RequestResponse
    resources:
      - group: ""
        resources: ["pods/exec", "pods/attach", "pods/portforward"]

  # Workload and service account mutations
  - level: Request
    verbs: ["create", "update", "patch", "delete"]
    resources:
      - group: ""
        resources: ["pods", "serviceaccounts"]
      - group: "apps"
        resources: ["daemonsets", "deployments", "statefulsets"]

  # Admission controller mutations
  - level: RequestResponse
    resources:
      - group: "admissionregistration.k8s.io"
        resources:
          - "mutatingwebhookconfigurations"
          - "validatingwebhookconfigurations"

  # Secrets and ConfigMaps: metadata only — never RequestResponse
  - level: Metadata
    verbs: ["create", "update", "get", "list", "watch"]
    resources:
      - group: ""
        resources: ["secrets", "configmaps"]

  # Suppress controller/scheduler read noise
  - level: None
    users:
      - "system:kube-controller-manager"
      - "system:kube-scheduler"
    verbs: ["get", "list", "watch"]

  - level: Metadata
```

Security-relevant audit log fields:

Field	Security Relevance
`user.username`	Initiating identity; `system:anonymous` = immediately actionable
`user.groups`	`system:masters` membership = cluster-admin equivalent
`sourceIPs`	Array of source addresses; external IP = out-of-cluster API call
`objectRef.resource`	Resource type targeted (pods, secrets, clusterrolebindings)
`objectRef.subresource`	`exec`, `portforward`, `token` are high-value subresources; `kubectl cp` appears as `pods/exec` using `tar` as the transfer mechanism — it does not generate a distinct subresource event
`verb`	`create` on RBAC resources = suspicious outside expected contexts
`responseStatus.code`	Chains of 403s indicate permission enumeration
`requestObject`	Full submitted spec; captures container-level `securityContext.privileged`

2.2 eBPF Runtime Telemetry

eBPF enables kernel-level event capture without requiring kernel module modification. For Kinsing's userland rootkit — which hooks userspace interfaces like `/proc` enumeration — kernel-level telemetry is essential: eBPF probes fire inside the kernel before userspace code has any opportunity to filter output.

eBPF is not the only approach to this class of visibility. `auditd` with `EXECVE` and socket syscall rules, kernel module-based agents, and commercial EDR products with privileged kernel drivers can provide comparable coverage. Each option carries different deployment complexity, maintenance cost, and performance profile.

How eBPF Captures Container Attacks

```
Kernel space                          Userspace
──────────────────────────────────────────────────

  syscall: execve()
    │
    ├─► eBPF kprobe fires (inside kernel)
    │     captures: pid, ppid, uid/gid,
    │       container cgroup ID → pod/namespace,
    │       argv[], filename
    │     writes to ring buffer
    │
    └─► Normal kernel execution continues
          (userland rootkit cannot intercept this path)

  Tetragon DaemonSet pod (userspace)
    ├─ Reads ring buffer
    ├─ Enriches with K8s metadata via Kubernetes API
    └─ Emits structured JSON event to stdout / gRPC
```

Tetragon vs. Falco vs. Tracee — Qualitative Comparison

Quantitative figures (CPU overhead percentages, exact kernel version thresholds) vary by kernel version, workload characteristics, probe type, and rule complexity. The values below are qualitative; consult each project's current official documentation before selecting a tool [R12] [R13] [R14].

Dimension	Tetragon (Cilium)	Falco	Tracee (Aqua)
Primary hook mechanism	eBPF kprobes and tracepoints	Modern eBPF probe (current default); legacy eBPF driver (deprecated); kernel module (supported for compatibility) [R12]	eBPF kprobes

Dimension	Tetragon (Cilium)	Falco	Tracee (Aqua)
Enforcement capability	Yes — `bpf_send_signal` can terminate a process or deny a syscall synchronously in-kernel	No — detect and alert only; cannot block a syscall	No — designed for forensic depth, not blocking
K8s metadata enrichment	Native via Kubernetes API (pod, namespace, image digest, labels)	Via plugin or driver configuration	Via enrichment module
Rule language	TracingPolicy CRD (YAML + eBPF selectors)	Falco rules DSL	Golang-based policies
Best operational fit	In-kernel enforcement; high-fidelity process and network lineage	Broad rule coverage; wide ecosystem integration	Deep forensic behavioral analysis; incident response

Recommended deployment: Falco and Tetragon are complementary. Falco provides the widest rule coverage and integrates readily with Fluent Bit, AWS Firelens, and standard SIEM pipeline tooling. Tetragon provides in-kernel enforcement at defined kill-chain steps where detect-only is insufficient.

Tetragon TracingPolicy: Detect nsenter Container Escape

```
# Detects nsenter escape into host namespace (T1611)
# Consult Tetragon documentation [R14] for kernel compatibility requirements
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: detect-nsenter-escape
spec:
  kprobes:
  - call: "sys_enter_setns"
    syscall: true
    args:
    - index: 0
      type: "int"
    - index: 1
      type: "int"
    selectors:
    - matchCapabilities:
      - type: Permitted
        isNamespaceCapability: false
        values:
        - "CAP_SYS_ADMIN"
      matchActions:
      - action: Sigkill
  - call: "sys_enter_execve"
    syscall: true
    selectors:
    - matchBinaries:
      - operator: In
        values:
        - "/usr/bin/nsenter"
        - "/bin/nsenter"
      matchActions:
      - action: Post
```

2.3 Telemetry Architecture by Platform

Self-Managed Kubernetes / Standard EKS (EC2 Node Groups)

DaemonSets deploy to EC2 worker nodes without restriction. Falco and Tetragon install via standard Helm charts. Kubernetes API audit logs must be explicitly configured via an audit policy file; the API server does not emit security-relevant events without this configuration.

On EKS, audit logs must be forwarded explicitly to CloudWatch Logs:

```
aws eks update-cluster-config \
  --name <cluster-name> \
  --logging '{"clusterLogging":[{"types":["api","audit","authenticator"],"enabled":true}]}'
```

AKS (Azure Kubernetes Service)

AKS surfaces control-plane audit logs through Azure Monitor Diagnostic Settings. Not all audit log categories are enabled by default; operators must explicitly enable the `kube-audit` and `kube-audit-admin` log categories in the cluster's Diagnostic Settings [R15]. Falco and Tetragon deploy as DaemonSets on AKS node pools without restriction.

GKE (Google Kubernetes Engine)

GKE surfaces Kubernetes audit logs through Cloud Audit Logs. Admin Activity logs (resource creation, deletion, RBAC mutations) are enabled by default. Data Access audit logs (pod exec sessions, secret reads) must be explicitly enabled per project and incur additional cost [R16]. GKE customizes the audit log format in ways that diverge from upstream Kubernetes; parsers targeting upstream Kubernetes must be adapted for GKE. Falco and Tetragon deploy as DaemonSets on GKE node pools.

EKS Auto Mode

EKS Auto Mode (GA November 2024) manages EC2 node provisioning via Karpenter and restricts the CNI to AWS VPC CNI [R17]. **DaemonSets are supported in EKS Auto Mode.** AWS Auto Mode nodes carry a taint that prevents non-system workloads from scheduling unless the DaemonSet explicitly tolerates it. Third-party security DaemonSets — Falco, Tetragon, and equivalent eBPF-based agents — must include the following toleration to deploy onto Auto Mode nodes [R17]:

```
tolerations:
  - key: "eks.amazonaws.com/compute-type"
    operator: "Equal"
    value: "auto"
    effect: "NoSchedule"
```

Without this toleration, the DaemonSet pods will remain in `Pending` state on Auto Mode nodes, creating a complete data-plane visibility gap. Validate DaemonSet scheduling against all node groups in a mixed Auto Mode / managed-node-group cluster before assuming telemetry coverage is uniform across nodes.

Cilium cannot be used as the cluster CNI in Auto Mode clusters; Hubble-based network observability that depends on Cilium as the primary CNI is therefore not available in this topology. Other observability approaches that do not require Cilium as the CNI may still be viable; verify against current AWS EKS Auto Mode documentation [R17].

AWS Fargate (Serverless Containers)

Fargate completely abstracts the underlying node. AWS documentation explicitly states the following constraints [R18]:

- **DaemonSets** — not supported
- **Privileged containers** — not supported
- **`HostNetwork`** — not supported
- **EKS Pod Identity** — not supported on Fargate; use IAM Roles for Service Accounts (IRSA) instead [R19]

Method	What it provides	Limitations
Application-side or pod-local telemetry	Process and network events within the pod — not standard Falco host-level coverage; no Falco node driver available on Fargate	No host-level visibility; no cross-pod correlation; no standard Falco driver model

Method	What it provides	Limitations
AWS GuardDuty for EKS	Managed runtime threat detection	Limited rule customization; AWS-specific findings only
EKS Audit Logs	Full API-plane visibility	No data-plane (runtime syscall) visibility
VPC Flow Logs	ENI-level network flows	No pod metadata; no process context

Fargate conclusion: The control plane (audit logs) remains fully visible. Data-plane runtime telemetry is severely constrained. Compensate with strict egress Network Policies, IMDS blocking (see Section 4.2), and document this gap explicitly in your threat model.

Pillar 3: Detection Engineering & "Parsers as Code"

3.1 Log Normalization and UDM Mapping

Raw Kubernetes audit events, Falco JSON alerts, and Tetragon process events use different schemas. Before correlation rules can operate across sources, both streams must be normalized to a common field model.

The mapping below is a **recommended normalization strategy** for organizations using Google Security Operations (Chronicle SIEM). It is not a universally applicable or automatically populated schema. For other SIEM products (Splunk, Elastic, Microsoft Sentinel), equivalent normalization pipelines use different field paths.

Fields are labeled as follows:
- **[STANDARD UDM]** — documented in Google's UDM field reference [R20]
- **[STANDARD UDM — REPEATED]** — documented as a repeated (multi-value) field [R20]
- **[PARSER-DEFINED]** — must be explicitly populated by a custom parser; not standard UDM taxonomy

Kubernetes Audit Field	UDM Field	Label	Sec
`user.username`	`principal.user.userid`	**[STANDARD UDM]**	API dete `sy`
`sourceIPs` (array)	`principal.ip`	**[STANDARD UDM — REPEATED]**	All exte clus
`objectRef.name`	`target.resource.name`	**[STANDARD UDM]**	Nan acte
`objectRef.resource`	`additional.fields["k8s_resource"]`	**[PARSER-DEFINED]**	Res clus etc.)
`objectRef.namespace`	`additional.fields["k8s_namespace"]`	**[PARSER-DEFINED]**	Log the
`verb`	`metadata.event_type` (parser-assigned constant)	**[PARSER-DEFINED mapping]**	Pars to U cons belo
`requestObject.spec.containers[].securityContext.privileged`	`additional.fields["container_privileged"]`	**[PARSER-DEFINED]**	Cap flag rule

Kubernetes Audit Field	UDM Field	Label	Secu
`requestObject.roleRef.name`	`additional.fields["roleRef_name"]`	**[PARSER-DEFINED]**	Rol Clus CRI mat dete adm

> ***`metadata.event_type` note:*** *UDM defines an `EVENT_TYPE` enum with values such as `USER_RESOURCE_CREATION` and `USER_RESOURCE_ACCESS`. A custom parser assigns these values based on Kubernetes `verb` and resource type (e.g., `verb=create + resource=pods` → `USER_RESOURCE_CREATION`). These are **parser-defined conventions**, not values that Chronicle populates automatically from Kubernetes audit events.*
>
> ***`sourceIPs` handling:*** *This is a JSON array in the Kubernetes audit log. In Chronicle YARA-L 2.0, `principal.ip` is a repeated field and can be queried using Chronicle's repeated-field syntax [R21]. A parser that renames `sourceIPs` directly to a single scalar field loses all but the first value.*
>
> ***GKE note:*** *GKE customizes the native Kubernetes audit log format. Parsers written for upstream Kubernetes must be adapted to handle GKE-specific field structures before they work correctly against GKE logs.*

Parser snippet [PSEUDOCODE — adapt to your pipeline (Logstash, Dataflow, custom ingestion)]:

```
[PSEUDOCODE]

if event.kind == "Event" and event.apiVersion matches "audit.k8s.io" {

  // Assign UDM event type based on verb and resource — this is a parser convention
  if event.objectRef.subresource == "exec" {
    udm.metadata.event_type = "USER_RESOURCE_ACCESS"      // parser-assigned constant
  } else if event.verb == "create" {
    udm.metadata.event_type = "USER_RESOURCE_CREATION"    // parser-assigned constant
  }

  // Principal — sourceIPs is an array; populate repeated field
  udm.principal.user.userid = event.user.username
  for ip in event.sourceIPs {
    udm.principal.ip.append(ip)                           // repeated field — all IPs
  }
  udm.network.http.user_agent = event.userAgent

  // Target — objectRef fields are nested, not top-level
  udm.target.resource.name            = event.objectRef.name
  udm.additional.fields["k8s_resource"]  = event.objectRef.resource
  udm.additional.fields["k8s_namespace"] = event.objectRef.namespace

  // ClusterRoleBinding — extract roleRef.name for RBAC backdoor detection (Rule 2)
  // requestObject.roleRef.name is only present on clusterrolebinding/rolebinding CREATE events.
  // Rule 2 matches additional.fields["roleRef_name"] == "cluster-admin"; this extraction
  // is the prerequisite that makes that match condition possible.
  if event.objectRef.resource == "clusterrolebindings" and event.verb == "create" {
    udm.additional.fields["roleRef_name"] = event.requestObject.roleRef.name
  }

  // Exec command captured from URI query parameters
  if event.objectRef.subresource == "exec" {
    cmd = extract_query_param(event.requestURI, "command")
    udm.additional.fields["exec_command"] = cmd
```

```
  }
}
```

3.2 Detection Rule 1 — Privileged Pod Creation (Backend-Dependent Sigma-Style Example)

Maps to: SCARLETEEL cryptominer staging; general privileged pod deployment in non-system namespaces
Source: Kubernetes Audit Logs

> ***Backend dependency — not a portable cross-backend Sigma rule:*** *This is a backend-dependent Sigma-style example, not universally deployable Sigma content. Kubernetes audit log fields such as* `requestObject.spec.containers.securityContext.privileged` *are not part of any standard Sigma field taxonomy. The dot-notation used here maps to nested JSON paths in the raw audit event; each backend handles array unrolling differently. This example has been developed against Elastic (nested field mappings) and Splunk (* `spath` *). It will not produce matches on backends that do not support array unrolling for these paths without backend-specific field mapping work. Treat it as a backend-specific starting point, not as a generic Sigma detection.*
>
> ***TeamTNT scope note:*** *TeamTNT RBAC Buster deploys its malicious DaemonSet inside* `kube-system`*. This rule excludes* `kube-system` *to suppress known-good security and CNI tooling, so it does* ***not*** *cover the canonical TeamTNT pattern. See* ***Rule 1b*** *(below) for* `kube-system`*-targeted DaemonSet detection.*

Portability note: Requires a SIEM backend that supports dot-notation and nested array field access against raw Kubernetes audit JSON. Container-level fields use flat dot-notation; the backend performs array unrolling. Test against your backend before promoting to production.

Assumptions / normalization dependencies:
- SIEM/backend must support dot-notation access to nested JSON audit event fields including `objectRef.*` and `requestObject.spec.*`
- **Array field handling:** Sigma does not support `[*]` wildcard notation in field keys. Detection fields for container-level flags use flat dot-notation (e.g., `requestObject.spec.containers.securityContext.privileged`). The containers field in a pod spec is a JSON array; each backend must be configured to unroll it before this rule can match. The two primary approaches are:
- **Elastic:** Index `requestObject` as a `nested` field type in the index mapping, then query using a `nested` query block so that Elastic evaluates field conditions within the same array element. A standard `keyword` or `object` mapping will flatten the array and cause incorrect cross-element matches. Reference: Elasticsearch nested field type documentation.
- **Splunk:** Use the `spath` command to extract the array path: `| spath input=requestObject path="spec.containers{}.securityContext.privileged" output=container_privileged` and then filter on `container_privileged=true`. The `{}` notation in `spath` iterates the array elements and outputs a multi-value field. This Sigma rule can only be translated to a working Splunk SPL query after this `spath` extraction step is added; the Sigma Splunk backend does not generate this automatically.
- Other SIEMs vary; confirm array-unrolling behavior in your specific backend before enabling in production.
- This rule targets raw `Pod` CREATE events in audit logs; DaemonSet/Deployment creates emit separate events with `objectRef.resource` set to `daemonsets` / `deployments` — see Rule 1b for the DaemonSet variant

```
title: Privileged or Host-Namespace Pod Created (Non-System Namespaces)
id: 7c3f9a1b-2e4d-4b8c-a5f2-1d9e7b3c6a4f
name: k8s_privileged_pod_creation
status: stable
description: >
  Detects creation of a Pod with a privileged container securityContext
  or with hostPID, hostNetwork, or hostIPC enabled, outside system namespaces.
  Covers SCARLETEEL staging and general privileged pod deployment in application
  namespaces. TeamTNT RBAC Buster deploys into kube-system — use Rule 1b for
  that pattern. Scope: raw Pod CREATE events only.
references:
```

```
  - https://www.aquasec.com/blog/leveraging-kubernetes-rbac-to-backdoor-clusters/
  - https://www.sysdig.com/blog/cloud-breach-terraform-data-theft
author: Detection Engineering
date: 2025/03/01
tags:
  - attack.privilege_escalation
  - attack.t1610
  - attack.t1611
  - attack.t1496
logsource:
  product: kubernetes
  service: audit
detection:
  selection_pod_create:
    verb: "create"
    objectRef.resource: "pods"
  filter_system_namespace:
    objectRef.namespace:
      - "kube-system"
      - "kube-public"
  # privileged is a per-container flag, not pod-level spec.securityContext
  # Note: Sigma does not support [*] array wildcard notation in field keys.
  # Use flat dot-notation; the target backend must perform array unrolling.
  # In Elastic: use nested queries; in Splunk: use spath. Test before deploying.
  selection_container_privileged:
    requestObject.spec.containers.securityContext.privileged: true
  selection_init_privileged:
    requestObject.spec.initContainers.securityContext.privileged: true
  # Each host-namespace flag is a separate selection to produce OR logic
  selection_hostPID:
    requestObject.spec.hostPID: true
  selection_hostNetwork:
    requestObject.spec.hostNetwork: true
  selection_hostIPC:
    requestObject.spec.hostIPC: true
  condition: >
    selection_pod_create
    and not filter_system_namespace
    and (
      selection_container_privileged
      or selection_init_privileged
      or selection_hostPID
      or selection_hostNetwork
      or selection_hostIPC
    )
falsepositives:
  - Security DaemonSets (Falco, Tetragon) in kube-system — excluded by namespace filter
  - CNI DaemonSets (Calico, Cilium) in kube-system — excluded by namespace filter
  - Privileged pods in non-system namespaces — investigate all remaining alerts
level: high
fields:
  - user.username
  - sourceIPs
  - objectRef.namespace
  - requestObject.spec.containers.image
  - requestObject.spec.containers.securityContext.privileged
```

3.2b Detection Rule 1b — Suspicious Privileged DaemonSet in kube-system (Backend-Dependent Sigma-Style Example)

Maps to: TeamTNT RBAC Buster DaemonSet deployment [R8]

Source: Kubernetes Audit Logs

> ***Why a separate rule:*** *TeamTNT creates its mining DaemonSet inside* `kube-system` *with a name mimicking* `kube-proxy`*. Rule 1 excludes that namespace entirely. This rule covers* `kube-system` *DaemonSet creates while suppressing known-good platform DaemonSets by name.* ***The allowlist below is illustrative only; rebuild it from your actual*** ***`kube-system`*** ***DaemonSet inventory before using this rule.*** *An unrecognized legitimate DaemonSet will alert.*

Portability note: The `filter_known_good` allowlist is environment-specific. Every newly deployed security or CNI DaemonSet in `kube-system` must be added promptly or this rule will fire on it. **Never copy this sample list as a reusable baseline.** Generate the authoritative allowlist from `kubectl get daemonsets -n kube-system` and revalidate after every cluster upgrade, CNI change, or security agent deployment.

Maintenance note: This is a high-maintenance allowlist rule. Every cluster upgrade, new security tool deployment, or CNI change is a potential source of false positives. **Deploy in Audit mode only until the allowlist is validated against your specific environment.** Ongoing maintenance is required; this is not a set-and-forget analytic. Rule 1b is an example of an allowlist-based detection approach for the most volatile namespace in the cluster — it is not a recommended reusable baseline across clusters and should not be ported without rebuilding the allowlist from scratch against the target environment.

> ***Backend dependency — not a portable cross-backend Sigma rule:*** *Like Rule 1, this is a backend-dependent Sigma-style example. The* `requestObject.spec.template.spec.*` *field paths are raw Kubernetes audit JSON paths, not standard Sigma field taxonomy. Array unrolling for container-level fields is backend-specific. This example targets Elastic and Splunk backends; other backends require field mapping validation before the rule can produce matches.*

Assumptions / normalization dependencies:
- `objectRef.resource: "daemonsets"` targets DaemonSet CREATE events; the audit log uses plural lowercase resource names
- DaemonSet container spec path is `requestObject.spec.template.spec.containers.securityContext.*` — not `requestObject.spec.containers.securityContext.*` (Pod-level path); Sigma uses flat dot-notation without `[*]` wildcards
- Host namespace flags for DaemonSets live at `requestObject.spec.template.spec.hostPID` etc.

```
title: Suspicious Privileged DaemonSet Created in kube-system
id: 9a2b1c4d-5e6f-4a7b-8c9d-0e1f2a3b4c5d
name: k8s_privileged_daemonset_kubesystem
status: experimental
description: >
  Detects a DaemonSet created in kube-system with privileged containers or
  host namespace flags that does not match known platform DaemonSet names.
  TeamTNT RBAC Buster creates such a DaemonSet named to mimic kube-proxy [R8].
  The filter_known_good list is illustrative only and must be rebuilt from the
  reader's actual kube-system DaemonSet inventory. Deploy in Audit mode only
  until the allowlist is validated. High-maintenance: any new DaemonSet in
  kube-system must be added promptly or it will alert.
references:
  - https://www.aquasec.com/blog/leveraging-kubernetes-rbac-to-backdoor-clusters/
author: Detection Engineering
date: 2025/03/01
tags:
  - attack.persistence
  - attack.t1053.007
  - attack.t1496
logsource:
  product: kubernetes
  service: audit
detection:
```

```
  selection_daemonset_create:
    verb: "create"
    objectRef.resource: "daemonsets"
    objectRef.namespace: "kube-system"
  filter_known_good:
    # ILLUSTRATIVE ONLY — rebuild from your actual kube-system DaemonSet inventory
    # Run: kubectl get daemonsets -n kube-system to enumerate yours
    # Do NOT include resource types deployed as Deployments (e.g. CoreDNS is a Deployment, not a Da
    objectRef.name:
      - "aws-node"
      - "kube-proxy"
      - "falco"
      - "tetragon"
      - "calico-node"
      - "cilium"
      - "aws-guardduty-agent"
      - "node-exporter"
      - "ebs-csi-node"
  # DaemonSet container spec lives under spec.template.spec — not spec directly
  # Note: [*] array wildcard notation is not valid Sigma syntax.
  # Use flat dot-notation; backend must support array unrolling (Elastic nested / Splunk spath).
  selection_privileged:
    requestObject.spec.template.spec.containers.securityContext.privileged: true
  selection_hostPID:
    requestObject.spec.template.spec.hostPID: true
  selection_hostNetwork:
    requestObject.spec.template.spec.hostNetwork: true
  selection_hostIPC:
    requestObject.spec.template.spec.hostIPC: true
  condition: >
    selection_daemonset_create
    and not filter_known_good
    and (
      selection_privileged
      or selection_hostPID
      or selection_hostNetwork
      or selection_hostIPC
    )
falsepositives:
  - Newly deployed security or CNI DaemonSets not yet added to filter_known_good — tune aggressivel
  - Emergency debugging DaemonSets — investigate all remaining alerts
level: high
fields:
  - user.username
  - sourceIPs
  - objectRef.name
  - requestObject.spec.template.spec.containers.image
  - requestObject.spec.template.spec.containers.securityContext.privileged
```

3.3 Detection Rule 2 — RBAC ClusterRoleBinding Backdoor (YARA-L)

Maps to: TeamTNT RBAC Buster persistent backdoor
Source: Kubernetes audit logs normalized to Chronicle UDM via custom parser

Critical dependency: The custom UDM parser described in Section 3.1 is not a minor implementation detail — it is the core engineering dependency that determines whether this rule is even possible. Until that parser exists, ingests data correctly, and populates every `additional.fields` key used below, this rule will produce zero matches. Do not treat the rule and the parser as separable.

Assumptions / normalization dependencies:

This is a **[PARSER-SPECIFIC EXAMPLE]**. It requires a custom UDM parser that populates the `additional.fields` keys and assigns `metadata.event_type` constants as described in Section 3.1. Fields using `additional.fields["key"]` syntax follow Chronicle's documented repeated-field access pattern [R21]. The rule uses only documented UDM fields and documented YARA-L 2.0 syntax [R22]. It will not function without the described parser.

Portability note: This is not portable Chronicle content; it depends on a custom parser contract. Do not deploy without first implementing the parser described in Section 3.1 and validating all `additional.fields` key names in your Chronicle instance against your actual parser output.

```
// [PARSER-SPECIFIC EXAMPLE]
// Requires custom K8s audit log UDM parser (see Section 3.1)
// Test in your Chronicle instance before promoting to production

rule K8s_RBAC_ClusterAdmin_Binding_Then_KubeSystem_Pod {
  meta:
    author      = "Detection Engineering"
    description = "ClusterRoleBinding granting cluster-admin created, followed within 5 minutes by
    severity    = "CRITICAL"
    mitre_attack = "T1098"

  events:
    // Event 1: ClusterRoleBinding create
    // metadata.event_type value is parser-assigned from verb=create
    $e1.metadata.event_type = "USER_RESOURCE_CREATION"        // parser-assigned constant
    $e1.metadata.product_name = "kubernetes"
    $e1.principal.user.userid = $actor                        // [STANDARD UDM]

    // objectRef fields extracted by parser into additional.fields
    $e1.additional.fields["k8s_resource"] = "clusterrolebindings"   // [PARSER-DEFINED]
    $e1.additional.fields["roleRef_name"] = "cluster-admin"          // [PARSER-DEFINED]

    // Event 2: Pod create in kube-system within 5 minutes
    $e2.metadata.event_type = "USER_RESOURCE_CREATION"        // parser-assigned constant
    $e2.metadata.product_name = "kubernetes"
    $e2.principal.user.userid = $actor                        // REQUIRED: bind $actor in e2 so mat
    $e2.additional.fields["k8s_resource"]  = "pods"           // [PARSER-DEFINED]
    $e2.additional.fields["k8s_namespace"] = "kube-system"    // [PARSER-DEFINED]

    // Cluster identifier — parser populates from cluster ARN or API server endpoint
    // Use a parser-defined additional.fields key rather than overloading principal.hostname
    $e1.additional.fields["cluster_id"] = $cluster_id        // [PARSER-DEFINED]
    $e2.additional.fields["cluster_id"] = $cluster_id        // [PARSER-DEFINED]

    // Temporal ordering: binding before pod creation
    $e1.metadata.event_timestamp.seconds < $e2.metadata.event_timestamp.seconds

  match:
    $actor, $cluster_id over 5m

  condition:
    $e1 and $e2

  outcome:
    $risk_score = max(95)
    $description = strings.concat(
      "cluster-admin binding by [", $actor,
      "] followed by pod create in kube-system"
    )
}
```

3.4 Detection Rule 3 — exec-then-Outbound / IMDS Access (Elastic EQL)

Maps to: SCARLETEEL credential exfiltration (3a), SCARLETEEL IMDS theft (3b), Kinsing mining pool connection (3a)
Source: Elastic ECS-normalized events from Falco/Tetragon + network agent

Assumptions / normalization dependencies:
- Events use Elastic Common Schema (ECS) field names: `process.name`, `process.working_directory`, `destination.ip`, `container.id`
- `container.id` must be populated consistently across process and network events by the ingestion agent; `container.name` is intentionally avoided because it is not unique across namespaces and nodes
- Rules 3a and 3b are separate to avoid a logic contradiction: Rule 3a excludes IMDS traffic; Rule 3b is the dedicated IMDS alert. Do not merge them.

Rule 3a — exec-then-outbound (excludes IMDS — covered by Rule 3b):

```
// Elastic EQL
// Fires when a suspicious process spawns in a container and the same
// container connects to a non-RFC1918, non-IMDS address within 60 seconds

sequence by container.id with maxspan=60s
  [process where event.type == "start"
   and container.id != null
   and process.name in ("curl", "wget", "python3", "python", "perl",
                        "ruby", "bash", "sh")
   and not process.parent.name in ("init", "tini", "dumb-init", "s6-svscan")
   and not process.working_directory in ("/app", "/srv", "/opt/app")]

  [network where event.type == "connection_attempted"
   and container.id != null
   and not cidr_match(destination.ip,
     "10.0.0.0/8",
     "172.16.0.0/12",
     "192.168.0.0/16",
     "169.254.0.0/16"     // IMDS range excluded here; use Rule 3b for IMDS alerts
   )
   and destination.port in (80, 443, 4444, 4445, 6666, 8888, 9999, 14444)]
```

Rule 3b — IMDS access from application pods (environment-tuned analytic pattern — not a generic production baseline):

```
// Elastic EQL
// Fires when any non-system pod connects to the cloud IMDS endpoint
// This is the SCARLETEEL Phase 3 cloud-credential theft signal
// Environment-tuned: namespace suppression must be validated against your cluster's
// actual workload layout before this rule is promoted to production.

network where event.type == "connection_attempted"
and container.id != null
and destination.ip == "169.254.169.254"
and kubernetes.namespace != null
and kubernetes.namespace not in ("kube-system", "kube-public")
```

> ***Suppression logic note:*** *The filter above uses `kubernetes.namespace` — a field populated by the Elastic Kubernetes integration from the pod's namespace metadata. Namespace-based suppression is harder to spoof than container labels: a label (`app.kubernetes.io/name`) is user-controlled and can be set to any value by an attacker with `pods/create` permission. Namespace identity requires that the pod actually be scheduled in `kube-system` or `kube-public`, which admission control (Section 4.1) can enforce independently.* ***Namespace suppression is still a simplification and not a***

> ***universal production discriminator.*** *Legitimate IMDS callers may exist outside* `kube-system` *and* `kube-public` *depending on cluster design — for example, a custom CNI DaemonSet deployed in a dedicated infrastructure namespace, a monitoring agent in* `monitoring`*, or a node management component in* `kube-node-lease`*. This rule must be tuned against the real workload identity and namespace layout of each target cluster before it is relied upon. Treat it as an environment-tuned pattern, not generic production content.*
>
> ***Legitimate IMDS callers in*** **`kube-system`** ***:*** *The* `aws-node` *(VPC CNI) and* `kube-proxy` *DaemonSets run in* `kube-system` *and legitimately call IMDS for node role credentials. This rule suppresses the entire namespace, covering those and any other platform-level agents. If your environment has no admission control blocking privileged pods in application namespaces, a compromised application pod can be re-scheduled into* `kube-system` *by an attacker with sufficient RBAC — namespace suppression is a complement to, not a replacement for, admission control.*
>
> ***Label-based alternative (simplified, not production-grade):*** *Filtering on* `container.labels["app.kubernetes.io/name"]` *narrows suppression to named pods but is trivially bypassed by any workload that self-assigns those labels. Use namespace-based suppression for production; reserve the label variant for lab demonstrations where the spoofing risk is acceptable.*
>
> ***Why two separate rules:*** *A sequence correlation (Rule 3a) is the right model for detecting dropper staging (exec then connect). A single-event alert (Rule 3b) is the right model for detecting IMDS access. Merging them forces a choice between excluding IMDS (creating a blind spot for SCARLETEEL Phase 3) or including it (flooding Rule 3a with false positives from routine IMDS health checks). Keep them separate.*

3.5 Detection Rule 4 — Privileged Pod then Host File Access (Sigma Correlation)

Maps to: CVE-2024-21626 exploitation, runc escape attempts (CVE-2025-31133, CVE-2025-52565, CVE-2025-52881)
Source: K8s audit log (control plane) + eBPF/auditd host file access event (data plane)

Portability note: This is a conceptual correlation pattern, not a drop-in deployable rule. It requires consistent `pod_name` enrichment across two fundamentally different telemetry planes (Kubernetes API audit and kernel eBPF/auditd). Many environments will need a dedicated cgroup-to-pod enrichment pipeline before this correlation produces reliable matches.

Assumptions / normalization dependencies:
- Requires two child Sigma rules: `k8s_privileged_pod_creation` (Rule 1 above) and a companion eBPF/auditd rule detecting sensitive host file reads from within a container context
- **Both child rules must emit a field named** **`pod_name`** for the `group-by` join to function; this requires parser-level enrichment that correlates the pod name from the audit event with the container/cgroup identity in the eBPF event
- **Cgroup-to-pod enrichment is a common failure point:** eBPF events identify containers by Linux cgroup ID, not Kubernetes pod name. A custom enrichment step — typically querying the container runtime socket or Kubernetes API — must map cgroup IDs to pod names before the `group-by` join can work. In practice, this enrichment fails silently: when it is missing or delayed, the correlation produces no matches for real attacks rather than false positives, making the gap invisible in routine testing. Validate the enrichment pipeline independently before relying on this rule for production coverage
- Sigma `temporal` type correlation fires when all referenced rules match events sharing the same `group-by` field value within the timespan; no `condition` threshold block is needed for `temporal` (it is required for count-based types such as `event_count`). Per the current Sigma Correlation Rules Specification, the `type`, `rules`, `group-by`, and `timespan` fields are nested under a top-level `correlation:` key — not at the document root.

```
# Sigma Correlation Rule
# Per current Sigma Correlation Rules Specification (sigmahq.io)
# type, rules, group-by, and timespan are nested under a top-level 'correlation:' key.
title: Privileged Pod Creation Followed by Host Sensitive File Access
name: k8s_privileged_pod_then_host_file_access
status: experimental
description: >
  Correlates a privileged pod creation (API plane) with sensitive host
  file access from inside a container (data plane), indicating a container
  escape attempt. Both child rules must emit a pod_name field for the
```

```
  temporal join. See normalization notes above.
tags:
  - attack.t1611
  - attack.privilege_escalation
correlation:
  type: temporal
  rules:
    - k8s_privileged_pod_creation        # Rule 1 name field
    - linux_container_host_file_access   # companion eBPF/auditd rule (see child rule below)
  group-by:
    - pod_name      # bare field name — must be populated in BOTH child rules by parser enrichment
  timespan: 2m
```

Companion child rule outline [PSEUDOCODE — adapt schema to your eBPF/auditd event fields]:

```
title: Sensitive Host File Access from Container Context
name: linux_container_host_file_access
logsource:
  product: linux
  service: tetragon    # or falco, auditd — adapt field names to your source
detection:
  selection:
    event.type: "file_open"
    file.path|startswith:
      - "/etc/shadow"
      - "/etc/passwd"
      - "/var/lib/kubelet"
      - "/proc/1/ns"
    container.id|exists: true    # event occurred inside a container context
    pod_name|exists: true        # parser must populate this for group-by join
  condition: selection
level: high
```

Pillar 4: Defensive Architecture & Hardening

4.1 Admission Controllers: Enforce Before Scheduling

Admission controllers evaluate workload specifications at API submission time, before any pod is scheduled. In `Enforce` mode they prevent entire classes of attack technique from succeeding. In `Audit` mode they only generate log records while attacks succeed — Audit mode is observability, not a security control.

> ***Prevention vs. detection boundary for TeamTNT in `kube-system`:*** *The sample Kyverno policy below uses broad namespace carve-outs for `kube-system` and `kube-public` to prevent cluster degradation. This means the policy does **not** prevent the canonical TeamTNT-in-`kube-system` case — a malicious DaemonSet deployed there is not blocked by this sample control. For that specific pattern, admission control is not the stopping control;* ***Rule 1b is the primary signal.*** *The sample hardening posture for TeamTNT in `kube-system` is detection-first, not prevention-first. Readers should understand this constraint explicitly rather than inferring that the policy covers all TeamTNT techniques.*

Control-to-technique mapping:

Attack technique	Admission controller block
TeamTNT DaemonSet with `hostPID: true` in `kube-system`	Admission control alone does not cleanly stop this pattern: the sample Kyverno policy broadly excludes `kube-system` for operational safety. Use **Rule 1b** (audit log detection) as the primary signal for malicious DaemonSets in `kube-system`.

Attack technique	Admission controller block
SCARLETEEL privileged container	Deny `securityContext.privileged: true` cluster-wide
Host filesystem traversal via `hostPath`	Deny all `hostPath` volumes
RBAC Buster cluster-admin binding	Deny ClusterRoleBinding to `cluster-admin` for non-approved subjects
Unvetted container image	Require images from approved registry with digest pin

> ***CVE-2024-21626 clarification:*** *Denying `hostPath` volumes does **not** mitigate CVE-2024-21626. That vulnerability exploits an internal runc file descriptor leak and does not require a `hostPath` volume mount. Denying `hostPath` mitigates attacks that use explicit host filesystem mounts to access node-level credentials, binaries, or container runtime sockets.*

Kyverno ClusterPolicy — Privileged, Host Namespace, hostPath, Registry

> ***Policy note:*** *The example below is an environment-sensitive starting point, not a drop-in production policy. System namespace handling uses **broad namespace carve-outs** (`kube-system`, `kube-public`) for operational safety — not a fine-grained allowlist. This is a deliberately conservative posture: it trades tighter enforcement in system namespaces for cluster survivability. A mature implementation would replace namespace-level exclusions with inventory-driven exceptions scoped to specific service accounts, controller names, or labels — but that requires a known-good workload inventory that this document cannot provide for your environment. Test in Audit mode against your specific cluster before switching to Enforce. Do not treat this example as equivalent to the YARA-L and Sigma-correlation rules — those are explicitly labeled parser-specific; this Kyverno policy is equally environment-specific.*

What this blocks: Pods, DaemonSets, Deployments, and StatefulSets that are privileged, use host namespaces, mount `hostPath` volumes, or reference images outside the approved registry — **in non-system namespaces**.
What this does not block: Kubelet-managed static pods; workloads already running at policy installation time; attacks that first obtain cluster-admin credentials before reaching the API server; **any workload in `kube-system` or `kube-public`** — those namespaces are broadly excluded for operational safety, which means a malicious DaemonSet placed in `kube-system` (TeamTNT pattern) is not blocked by this policy. Use Rule 1b for that detection.
Platform caveats: Policy applies on API server admission. EKS Auto Mode, AKS, and GKE all run the API server; this policy is applicable on all of them. Fargate restrictions (no privileged, no hostNetwork) are enforced by the platform independently of this policy.

> ***Why broad namespace carve-outs rather than a strict allowlist:*** *Platform DaemonSets — CNI plugins (`aws-node`, `calico-node`, `cilium`), `kube-proxy`, and security agents (Falco, Tetragon) — legitimately require privileged containers, host namespaces, `hostPath` volumes, and images from upstream registries. A strict allowlist that names specific controllers by service account or label requires complete, maintained inventory of every platform component. The broad namespace exclusion below is an operational safety valve that prevents cluster degradation; it is not equivalent to "only known-good workloads are permitted in kube-system." Accept that tradeoff explicitly in your threat model.*

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: pod-security-baseline
  annotations:
```

```
    policies.kyverno.io/title: Deny Privileged, Host-Namespace, hostPath, and Unapproved-Registry W
    policies.kyverno.io/severity: critical
spec:
  validationFailureAction: Enforce
  background: true
  rules:

    # Privileged containers — Pod
    # spec.containers[].securityContext.privileged is the correct path for Pods
    - name: deny-privileged-pod
      match:
        any:
        - resources:
            kinds: ["Pod"]
      exclude:
        any:
        - resources:
            namespaces: ["kube-system"]
            selector:
              matchLabels:
                security.io/managed: "platform"
      validate:
        message: "Privileged containers are not permitted."
        pattern:
          spec:
            =(initContainers):
              - =(securityContext):
                  =(privileged): false
            containers:
              - =(securityContext):
                  =(privileged): false

    # Privileged containers — workload controllers
    # Container spec is at spec.template.spec, not spec, for DaemonSet/Deployment/StatefulSet
    # CRITICAL: kube-system must be excluded. CNI plugins (aws-node, calico-node, cilium)
    # and platform agents legitimately require privileged containers. Enforcing this rule
    # against kube-system without exclusions will break CNI and cause cluster degradation.
    - name: deny-privileged-controllers
      match:
        any:
        - resources:
            kinds: ["DaemonSet", "Deployment", "StatefulSet", "ReplicaSet"]
      exclude:
        any:
        - resources:
            namespaces:
              - "kube-system"
              - "kube-public"
      validate:
        message: "Privileged containers are not permitted in workload controllers."
        pattern:
          spec:
            template:
              spec:
                =(initContainers):
                  - =(securityContext):
                      =(privileged): false
                containers:
                  - =(securityContext):
                      =(privileged): false

    # Host namespace access — Pod
    - name: deny-host-namespaces-pod
```

```
    match:
      any:
      - resources:
          kinds: ["Pod"]
    validate:
      message: "hostPID, hostNetwork, and hostIPC are not permitted."
      pattern:
        spec:
          =(hostPID): false
          =(hostNetwork): false
          =(hostIPC): false

  # Host namespace access — workload controllers
  # CRITICAL: kube-system must be excluded. Platform DaemonSets (kube-proxy, aws-node,
  # calico-node, cilium) legitimately require hostNetwork: true. Enforcing without
  # exclusions will prevent platform DaemonSets from deploying or updating.
  - name: deny-host-namespaces-controllers
    match:
      any:
      - resources:
          kinds: ["DaemonSet", "Deployment", "StatefulSet"]
    exclude:
      any:
      - resources:
          namespaces:
            - "kube-system"
            - "kube-public"
    validate:
      message: "hostPID, hostNetwork, and hostIPC are not permitted in workload controllers."
      pattern:
        spec:
          template:
            spec:
              =(hostPID): false
              =(hostNetwork): false
              =(hostIPC): false

  # hostPath volumes — Pod
  # Blocks host filesystem mount abuse; does NOT mitigate CVE-2024-21626
  - name: deny-hostpath-pod
    match:
      any:
      - resources:
          kinds: ["Pod"]
    validate:
      cel:
        expressions:
          - expression: >
              !has(object.spec.volumes) ||
              object.spec.volumes.all(v, !has(v.hostPath))
            message: "hostPath volumes are not permitted."

  # hostPath volumes — workload controllers
  # spec.template.spec.volumes is the correct path for controllers
  # CRITICAL: kube-system must be excluded. Node-level agents (aws-node, ebs-csi-node,
  # Falco, Tetragon) legitimately mount host paths. Enforcing without exclusions will
  # break these agents and remove critical security and CNI tooling from all nodes.
  - name: deny-hostpath-controllers
    match:
      any:
      - resources:
          kinds: ["DaemonSet", "Deployment", "StatefulSet"]
    exclude:
```

```
        any:
        - resources:
            namespaces:
              - "kube-system"
              - "kube-public"
      validate:
        cel:
          expressions:
            - expression: >
                !has(object.spec.template.spec.volumes) ||
                object.spec.template.spec.volumes.all(v, !has(v.hostPath))
              message: "hostPath volumes are not permitted in workload controllers."

    # Approved registry with digest pin — Pod
    # CRITICAL: kube-system and kube-public must be excluded.
    # Platform components (CoreDNS, kube-proxy, aws-node) pull from upstream registries
    # (registry.k8s.io, ECR public) that do not match the internal registry pattern.
    # Enforcing without exclusions will block node provisioning and cluster upgrades.
    - name: require-approved-registry-pod
      match:
        any:
        - resources:
            kinds: ["Pod"]
      exclude:
        any:
        - resources:
            namespaces:
              - "kube-system"
              - "kube-public"
      validate:
        message: "Image must be from the approved registry with a digest pin."
        foreach:
        # containers, initContainers, and ephemeralContainers are all validated.
        # Omitting any one of these lists leaves a bypass path: an attacker can
        # run an unapproved image as an initContainer or ephemeralContainer and
        # completely circumvent the registry constraint.
        - list: "request.object.spec.containers"
          pattern:
            image: "registry.company.internal/*@sha256:*"
        - list: "request.object.spec.initContainers"
          pattern:
            image: "registry.company.internal/*@sha256:*"
        - list: "request.object.spec.ephemeralContainers"
          pattern:
            image: "registry.company.internal/*@sha256:*"

    # Approved registry with digest pin — workload controllers
    # CRITICAL: same kube-system exclusion required as for the Pod rule above.
    - name: require-approved-registry-controllers
      match:
        any:
        - resources:
            kinds: ["DaemonSet", "Deployment", "StatefulSet"]
      exclude:
        any:
        - resources:
            namespaces:
              - "kube-system"
              - "kube-public"
      validate:
        message: "Image must be from the approved registry with a digest pin."
        foreach:
        # All three container lifecycle lists must be validated — see note above.
```

```
        - list: "request.object.spec.template.spec.containers"
          pattern:
            image: "registry.company.internal/*@sha256:*"
        - list: "request.object.spec.template.spec.initContainers"
          pattern:
            image: "registry.company.internal/*@sha256:*"
        - list: "request.object.spec.template.spec.ephemeralContainers"
          pattern:
            image: "registry.company.internal/*@sha256:*"
```

OPA Gatekeeper — Restrict ClusterRoleBinding to cluster-admin

What this blocks: Any ClusterRoleBinding that grants `cluster-admin` to a subject not in the approved list. Each unapproved subject generates a separate violation, preventing a mixed approved/unapproved subject set from passing.
What this does not block: Privilege escalation paths that do not create cluster-admin bindings; bindings created before this policy was installed; direct etcd or kubelet manipulation.
Platform caveats: Gatekeeper must be installed and operating in Enforce mode. Applies equally across EKS, AKS, and GKE managed control planes.

> ***Why `subject.kind` must be validated alongside `subject.name`:*** *Kubernetes RBAC subject names are scoped to their `kind` — they are not globally unique. An attacker can create a `ServiceAccount` named `system:kube-controller-manager` in any namespace and bind it to `cluster-admin`. A policy that only checks `subject.name` will treat this attacker-controlled ServiceAccount as approved. The Rego below enforces both `subject.kind` and `subject.name` so that the allowlist cannot be bypassed by name collision across kinds.*

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sblockclusteradminbinding
spec:
  crd:
    spec:
      names:
        kind: K8sBlockClusterAdminBinding
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sblockclusteradminbinding

        # Iterate over every subject; generate a violation for EACH unapproved one.
        # A binding containing one approved subject and one unapproved subject
        # still produces a violation — preventing partial-bypass of the allowlist.
        violation[{"msg": msg}] {
          input.review.kind.kind == "ClusterRoleBinding"
          input.review.object.roleRef.name == "cluster-admin"
          subject := input.review.object.subjects[_]
          not is_approved_subject(subject)
          msg := sprintf(
            "ClusterRoleBinding to cluster-admin is not permitted for %v '%v'",
            [subject.kind, subject.name]
          )
        }

        # Each approved subject is matched on BOTH kind AND name.
        # Matching name alone allows a ServiceAccount named "system:kube-controller-manager"
        # to bypass this policy — subject names are not globally unique across kinds.
```

```
        is_approved_subject(subject) {
          subject.kind == "User"
          subject.name == "system:kube-controller-manager"
        }
        is_approved_subject(subject) {
          subject.kind == "User"
          subject.name == "system:kube-scheduler"
        }
        is_approved_subject(subject) {
          subject.kind == "User"
          subject.name == "eks:cluster-bootstrap"
        }
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sBlockClusterAdminBinding
metadata:
  name: block-cluster-admin-binding
spec:
  match:
    kinds:
      - apiGroups: ["rbac.authorization.k8s.io"]
        kinds: ["ClusterRoleBinding"]
```

4.2 Network Policy: Block IMDS and Constrain Egress

What this blocks: EC2 IMDS credential theft from application pods on EC2-backed nodes (SCARLETEEL Phase 3); cryptomining pool connections; C2 callbacks and exfiltration to arbitrary public IPs.
What this does not block: Attacks completing entirely within the cluster without external connectivity; IMDS access from pods with `hostNetwork: true` (prevent those via admission control); lateral movement using already-obtained credentials.
Platform caveats: Network policies require a CNI that enforces them (Calico, Cilium, VPC CNI with policy support). EKS Auto Mode uses VPC CNI; verify Network Policy enforcement capability for your version.

> ***Fargate IMDS architecture note:*** *EKS Fargate does* ***not*** *use the EC2 Instance Metadata Service (* `169.254.169.254` *) — there is no local metadata IP endpoint involved in credential delivery. EKS Fargate delivers AWS credentials exclusively through* ***IAM Roles for Service Accounts (IRSA)*** *via OIDC federation. At pod startup, the Fargate data plane injects two environment variables (* `AWS_WEB_IDENTITY_TOKEN_FILE` *and* `AWS_ROLE_ARN` *) and mounts a projected web identity token file. The AWS SDK reads those variables and calls the regional* ***STS endpoint*** *(e.g.,* `sts.amazonaws.com` *) directly over HTTPS to exchange the projected token for temporary IAM credentials. No link-local metadata IP is involved at any step. A network policy blocking* `169.254.169.254` *has zero effect on EKS Fargate credential delivery. For Fargate, credential hygiene means scoping each IRSA IAM role to the minimum required permissions, auditing OIDC trust policies for overly broad conditions, and rotating or bounding token lifetimes via projected volume* `expirationSeconds` *— not network-layer IMDS blocking.*
>
> ***Detection opportunity on Fargate:*** *Although Fargate does not use IMDS, attacker tooling executing inside a compromised Fargate container will still attempt to reach* `169.254.169.254` *— generic cloud-exploitation frameworks probe it unconditionally as part of credential-harvesting routines. These connection attempts will fail at the network layer, but the failed/blocked flows are recorded in* ***VPC Flow Logs*** *as* `REJECT` *records toward* `169.254.169.254`*. Monitoring VPC Flow Logs for outbound REJECT events to that destination from Fargate pod ENIs is a* ***high-fidelity detection signal****: legitimate Fargate workloads have no reason to initiate connections to that address. A* `REJECT` *record toward* `169.254.169.254` *from a Fargate task ENI is effectively a near-zero false-positive indicator of attacker reconnaissance tooling executing inside the container.*

Block EC2 IMDS for Non-System Pods (EC2 nodes only — not applicable to Fargate)

> ***Scope:*** *This policy blocks access to the EC2 Instance Metadata Service at* `169.254.169.254`*. It applies only to pods running on EC2 worker nodes. EKS Fargate pods do not use any local metadata IP endpoint — credentials are delivered via IRSA/OIDC with direct STS calls — and are therefore unaffected by and do not benefit from this rule.*

```
apiVersion: cilium.io/v2
kind: CiliumClusterwideNetworkPolicy
metadata:
  name: block-imds-access
spec:
  endpointSelector:
    matchExpressions:
      - key: k8s:io.kubernetes.pod.namespace
        operator: NotIn
        values:
          - kube-system    # aws-node and similar require IMDS access
  egressDeny:
    - toCIDRSet:
        - cidr: 169.254.169.254/32
```

EKS Pod Identity and IMDS interaction: EKS Pod Identity reduces the blast radius of IMDS credential theft by issuing credentials scoped to the requesting pod's Service Account rather than the full node IAM role [R19]. However, AWS documents that Pod Identity operates by intercepting IMDS calls through the SDK credential chain. Pods using `hostNetwork: true` retain direct IMDS access because they share the host network namespace [R19]. Pod Identity also does not eliminate the underlying IMDS endpoint for other callers. It reduces exposure but does not make IMDS harvesting impossible. The network-level IMDS block above is a complementary control and should be applied regardless.

> ***Fargate:*** *EKS Pod Identity is not supported on Fargate [R19]. Use IRSA (IAM Roles for Service Accounts) for Fargate workloads requiring AWS IAM access.*

Namespace Default-Deny with Approved Egress

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-approved-egress
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

```
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: internal-services
      ports:
        - protocol: TCP
          port: 8080
```

4.3 Seccomp and AppArmor: Syscall-Level Restriction

What this blocks: `setuid` / `setgid` abuse via `allowPrivilegeEscalation`; dropper file writes to the container root filesystem. `RuntimeDefault` Seccomp reduces the available syscall surface and may reduce exploitability of some container escape paths; the specific interaction with any individual CVE requires per-CVE analysis against the exact allowed syscall profile. The primary mitigation for the runc CVEs covered in this paper is patching runc, not Seccomp configuration.
What this does not block: Vulnerabilities that operate within the allowed syscall set; attacks by processes that have already escaped container isolation; application-layer vulnerabilities.
Platform caveats: Seccomp `RuntimeDefault` is available on EKS (EC2 and Auto Mode), AKS, and GKE. Fargate does not support privileged containers but its Seccomp behavior should be validated against AWS Fargate documentation for the version in use.

2025 runc CVEs — Prerequisites and Scope

All three vulnerabilities require an attacker to already have code execution inside a container. None are remotely exploitable in isolation [R6].

CVE	Mechanism	Prerequisites
CVE-2025-31133	Bind-mount masking race condition	Local container context; specific bind-mount configuration required [R6]
CVE-2025-52565	`/dev/console` device exploitation	Local container context; container with `/dev/console` accessible [R6]
CVE-2025-52881	Shared-mount propagation race condition	Local container context; mount namespace conditions required [R6]

Applying `RuntimeDefault` Seccomp reduces attack surface and may reduce exploitability for some of these escape paths; the specific syscall-level benefit for each CVE has not been independently verified in this paper. The primary mitigation for all three is upgrading runc to a patched version.

```
# Kubernetes >= v1.30: AppArmor is specified via securityContext.appArmorProfile [R24]
# Kubernetes < v1.30: use the annotation form instead:
#   metadata.annotations:
#     container.apparmor.security.beta.kubernetes.io/<container-name>: runtime/default
apiVersion: v1
kind: Pod
metadata:
  name: hardened-workload
  namespace: production
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
    appArmorProfile:          # Kubernetes v1.30+ [R24]; use annotation on older clusters
      type: RuntimeDefault
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
```

```
  containers:
  - name: app
    image: registry.company.internal/app@sha256:<digest>
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true    # Blocks file-based dropper writes
      capabilities:
        drop:
          - ALL
        add:
          - NET_BIND_SERVICE          # Add only what the application actually requires
    volumeMounts:
    - name: tmp-vol
      mountPath: /tmp
  volumes:
  - name: tmp-vol
    emptyDir: {}
```

4.4 RBAC Hardening: Minimize Service Account Token Exposure

What this blocks: Automatic credential injection into every pod — the SA token theft vector used by SCARLETEEL Phase 2 and Slow Pisces; eliminates the well-known credential path at `/var/run/secrets/kubernetes.io/serviceaccount/token` for pods where API access is not required.
What this does not block: Attackers who separately request a new token via the API server after obtaining API access through another path; SA tokens stored in environment variables or Secrets rather than auto-mounted.
Platform caveats: Applies equally across all Kubernetes distributions and managed services. Projected tokens with short expiration are supported on EKS, AKS, GKE, and Fargate.

The structural problem: By default, the ServiceAccount admission controller injects a credential volume into every pod at `/var/run/secrets/kubernetes.io/serviceaccount/token`. In modern Kubernetes (v1.22+), this is a TokenRequest-backed projected token with a bounded lifetime and automatic rotation [R25] — not the older static Secret-based token used in pre-v1.22 behavior. Either way, any attacker with code execution inside a pod automatically has a usable credential to the kube-apiserver at a well-known path — unless `automountServiceAccountToken: false` is set on the ServiceAccount or the Pod. Microsoft's April 2025 analysis found that **51% of workload identities were completely inactive** [R1], making dormant SAs preferred persistence anchors for actors like Slow Pisces — because they generate no behavioral anomaly alerts.

Disable Auto-Mount on the Default ServiceAccount

```
# Apply to every namespace's default ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: default
  namespace: production
automountServiceAccountToken: false
```

Short-Lived Projected Tokens for Pods That Require API Access

> ***Token lifetime note:*** *`serviceAccountToken` projected volumes default to `expirationSeconds: 3600` (1 hour) with a minimum of 600 seconds (10 minutes) [R25]. The kubelet automatically rotates these tokens before expiry. This is distinct from the older static ServiceAccount Secrets (now deprecated), which generated long-lived non-expiring tokens. Do not conflate the two models when assessing token exposure risk.*

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: needs-api-access
spec:
  serviceAccountName: minimal-api-sa
  automountServiceAccountToken: false
  volumes:
  - name: api-token
    projected:
      sources:
      - serviceAccountToken:
          path: token
          expirationSeconds: 3600       # explicit 1-hour TTL; projected SA token default is 3600s
          audience: "https://kubernetes.default.svc"
  containers:
  - name: app
    image: registry.company.internal/app@sha256:<digest>
    volumeMounts:
    - name: api-token
      mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      readOnly: true
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: minimal-api-role
  namespace: production
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  resourceNames: ["app-config"]    # Named resource access only — not a wildcard
  verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: minimal-api-binding
  namespace: production
subjects:
- kind: ServiceAccount
  name: minimal-api-sa
  namespace: production
roleRef:
  kind: Role
  name: minimal-api-role
  apiGroup: rbac.authorization.k8s.io
```

Audit Queries to Surface Live Risk

```bash
# Find ClusterRoleBindings granting cluster-admin to non-system ServiceAccounts
kubectl get clusterrolebindings -o json | jq -r '
  .items[] |
  select(.roleRef.name == "cluster-admin") |
  .subjects[]? |
  select(.kind == "ServiceAccount") |
  select(.namespace != "kube-system") |
  "RISKY: \(.namespace)/\(.name)"
'
```

```
# Find pods with auto-mounted SA tokens
kubectl get pods -A -o json | jq -r '
  .items[] |
  select((.spec.automountServiceAccountToken // true) == true) |
  "\(.metadata.namespace)/\(.metadata.name) SA=\(.spec.serviceAccountName // "default")"
'
```

Takeaways for the SOC

1. Enable Kubernetes audit logs and confirm they reach your SIEM before any other action.
The default enablement state differs by platform: EKS requires explicit CloudWatch configuration; AKS requires enabling `kube-audit` in Diagnostic Settings; GKE requires explicit Data Access log enablement for sensitive operations. Without logs covering `serviceaccounts/token` creates at `RequestResponse` level, SA token theft as documented in Slow Pisces and SCARLETEEL is invisible.

2. Alert on `system:anonymous` in audit logs with zero suppression.
Any event with `user.username == "system:anonymous"` is either misconfiguration or active exploitation. This is TeamTNT RBAC Buster's entry condition. On a correctly configured cluster, the false positive rate is zero.

3. Treat ClusterRoleBinding creation to `cluster-admin` as a P1 incident with an automated response playbook.
RBAC Buster creates bindings that survive misconfiguration remediation. When this alert fires: suspend associated credentials immediately, audit all binding subjects, and review all pods created in the following 10 minutes.

4. Correlate pod exec (audit log) with outbound network connections (Rule 3a) as a sequence within 60 seconds.
Neither signal alone produces high-confidence detections. A `curl` inside a container is normal; an outbound connection to a public IP is normal. Their sequence from the same container is the behavioral fingerprint of dropper staging in SCARLETEEL and Kinsing campaigns. Keep IMDS as a separate single-event alert (Rule 3b).

5. Deploy eBPF-based or equivalent kernel-level telemetry on every non-Fargate node.
Kinsing's userland rootkit conceals processes from userspace enumeration. Kernel-level telemetry fires before userspace filtering. On Fargate, DaemonSets and privileged containers are unsupported by design [R18]; EKS Pod Identity is also not available [R19]. Document the data-plane visibility gap and compensate with strict egress Network Policies and IMDS blocking.

6. Set `automountServiceAccountToken: false` on the default ServiceAccount in every namespace.
This removes one default, well-known credential path from pods that do not need API access. It is a useful hardening step — the auto-mounted token is a primary initial pivot in SCARLETEEL Phase 2, Kinsing lateral movement, and Slow Pisces cluster pivoting — but it is not a decisive control over pod credential exposure. Attackers who obtain API server access through another route can still request a new token via the TokenRequest API, extract credentials mounted as Secrets, pull them from CI/CD systems, or obtain them through cloud IAM paths. Setting this field reduces the attack surface for one specific, well-documented vector; it does not close credential exposure as a class of risk. See the `What this does not block` note in Section 4.4.

7. Run admission controllers in `Enforce` mode before workloads reach production.
Audit mode allows attacks to succeed and produces a log. The Kyverno policies in Section 4.1 deny privileged container patterns (SCARLETEEL), host namespace access, and `hostPath` volume abuse in non-system namespaces. They do **not** prevent the canonical TeamTNT-in-`kube-system` pattern — the sample policy excludes `kube-system` for operational safety. For that specific pattern, Rule 1b (audit log DaemonSet detection) is the primary signal; admission control is not the stopping control. Use separate rules for Pods and workload controllers — the container spec path differs (`spec.containers` vs `spec.template.spec.containers`).

8. Enumerate and revoke inactive ServiceAccount identities quarterly.
Microsoft's April 2025 analysis found 51% of workload identities completely inactive [R1]. Dormant SAs are preferred by Slow Pisces-style actors because there is no behavioral baseline to deviate from — they trigger no anomaly alerts. Query for SAs with no bound pods and no recent API server activity; revoke or scope-limit them.

9. Block IMDS access at the CNI layer for all non-system pods.
SCARLETEEL's entire cross-account pivot relied on unrestricted IMDS access from a single compromised container. A CNI

network policy blocking `169.254.169.254/32` for non-system pods severs this vector independent of application code. Note: pods with `hostNetwork: true` retain direct IMDS access through the host network namespace; prevent those via admission control (Section 4.1).

10. Test your detection logic quarterly against actual attacker technique artifacts.
In a dedicated test namespace with all egress blocked: submit a pod manifest with `hostPID: true`. Verify that the admission controller rejects it and emits an audit event, that your audit log captures the attempt, and that your SIEM alert fires within SLA. Detection rules not exercised against real event patterns degrade silently.

MITRE ATT&CK Mapping

Technique	ID	Actor(s)	Primary Defensive Control
Exploit Public-Facing Application	T1190	SCARLETEEL, Kinsing, React2Shell	Rapid patching; runtime immutability
Deploy Container	T1610	TeamTNT, SCARLETEEL	Admission controller Enforce mode
Container Administration Command	T1609	All actors	Audit logging of `pods/exec`; alert on exec from unexpected principals
Escape to Host	T1611	CVE-2024-21626, nsenter	runc upgrade (primary); Seccomp `RuntimeDefault` (partial surface reduction)
Steal Application Access Token	T1528	SCARLETEEL, Slow Pisces	`automountServiceAccountToken: false`
Unsecured Credentials: Files	T1552.001	SCARLETEEL	`readOnlyRootFilesystem`; deny `hostPath`
Unsecured Credentials: Cloud Metadata	T1552.005	SCARLETEEL, Kinsing	Block IMDS at CNI; EKS Pod Identity / IRSA
Valid Accounts: Cloud Accounts	T1078.004	SCARLETEEL, Slow Pisces	CloudTrail anomaly detection; least-privilege IAM
Account Manipulation	T1098	TeamTNT RBAC Buster	Alert on `ClusterRoleBinding` to `cluster-admin`; OPA Gatekeeper deny
Scheduled Task/Job: Container	T1053.007	TeamTNT	Alert on DaemonSet creation; deny `hostPID` / `hostNetwork` via Kyverno
Transfer Data to Cloud Account	T1537	SCARLETEEL	Egress `NetworkPolicy`; S3 access logging
Resource Hijacking	T1496	Kinsing, TeamTNT, LLMjacking	Admission control; resource quotas; AI API usage monitoring
Phishing: Spearphishing	T1566	Slow Pisces	Security awareness training; MFA on email; sandbox email attachment inspection
Masquerading	T1036	TeamTNT	Registry allowlist; image digest pinning
Rootkit	T1014	Kinsing	Kernel-level telemetry (eBPF or equivalent)
Indicator Removal: Clear History	T1070.003	SCARLETEEL	K8s audit logs are independent of shell history
Valid Accounts: Default Accounts	T1078.001	FAMOUS CHOLLIMA	MFA on all remote access; zero-trust network access for developers

---

References

[R1] Understanding the threat landscape for Kubernetes and containerized assets | Microsoft Security Blog, April 23, 2025

[R2] SCARLETEEL: Operation leveraging Terraform, Kubernetes, and AWS for data theft | Sysdig TRT

[R3] Understanding Current Threats to Kubernetes Environments | Palo Alto Unit 42

[R4] 2025 Threat Hunting Report | CrowdStrike

[R5] LLMjacking: Stolen Cloud Credentials Used in New AI Attack | Sysdig TRT

[R6] NVD advisories: CVE-2025-31133 · CVE-2025-52565 · CVE-2025-52881

[R7] CVE-2024-21626 Impact, Exploitability, and Mitigation | Wiz

[R8] First-Ever Attack Leveraging Kubernetes RBAC to Backdoor Clusters | Aqua Nautilus

[R9] Initial Access Techniques in Kubernetes Environments Used by Kinsing Malware | Microsoft Defender for Cloud Blog

[R10] Defending against CVE-2025-55182 (React2Shell) in React Server Components | Microsoft Security Blog

[R11] CIS Google Kubernetes Engine (GKE) Benchmark v1.5.0 | CIS

[R12] Falco — Setup and Download | falco.org

[R13] Tracee Documentation | Aqua Security

[R14] Tetragon Documentation | tetragon.io

[R15] Monitor Azure Kubernetes Service with Azure Monitor | Microsoft Learn

[R16] Configure audit logs for Google Kubernetes Engine | Google Cloud

[R17] Getting started with Amazon EKS Auto Mode | AWS Containers Blog

[R18] AWS Fargate for Amazon EKS — Considerations | AWS Documentation

[R19] How EKS Pod Identity works | AWS Documentation

[R20] UDM Field List | Google Security Operations | Google Cloud

[R21] Repeated Fields in YARA-L 2.0 | Google Security Operations | Google Cloud

[R22] YARA-L 2.0 Language Syntax | Google Security Operations | Google Cloud

[R23] Sigma Correlation Rules Specification | SigmaHQ

[R24] Restrict a Container's Access to Resources with AppArmor | Kubernetes — documents the `securityContext.appArmorProfile` field (v1.30+) and the deprecated annotation form for earlier versions

[R25] Projected Volumes | Kubernetes — documents `serviceAccountToken` projected volume defaults: `expirationSeconds` defaults to 3600 (1 hour), minimum 600 (10 minutes); kubelet rotates the token automatically before expiry

↑ Back to top